

State-Based Process Monitoring in Traditional Event Logs

Contents

1	Paper Foundations	2
2	Sliding Windows	5
3	PCA – Compress	6
4	Self-Organizing Maps	8
5	Boundary Conditions	9
6	Three State Types for Traditional Event Logs	11
6.1	Intra-Case State	11
6.2	Resource State	12
6.3	Inter-Case State	13
7	Combined State Signal & Concept Drift Detection	14
8	Implementation Roadmap	16

1 Paper Foundations

Task 1: Paper notation instantiations

Use the tiny running example below to instantiate the paper notation. For each item (a)–(e), produce three outputs: a concrete instantiation using the example, the meaning of the instantiated object, and one short sentence about how it is used in the window–compress–map pipeline. Do not only define the symbols; write actual sets, maps, vectors, windows, or state assignments.

Event	Activity	Time	Related objects	Attributes
e_1	Create	08:00	$\{o_1 : \text{Order}, i_1 : \text{Item}\}$	amount = 100, channel = web
e_2	Ship	09:00	$\{o_1 : \text{Order}, i_1 : \text{Item}\}$	amount = 100, channel = web
e_3	Create	09:10	$\{o_2 : \text{Order}, i_2 : \text{Item}, i_3 : \text{Item}\}$	amount = 240, channel = phone

Use, where needed, $A = \{\text{Create}, \text{Ship}\}$, $OT = \{\text{Order}, \text{Item}\}$, $K_{\text{num}} = \{\text{amount}\}$, and $K_{\text{str}} = \{\text{channel}\}$. For PCA and SOM, you may choose a small numerical example, but the numbers must be dimensionally consistent with the formulas.

(a) **OCEL.** Let U_σ denote the universe of all strings. An object-centric event log is

$$(E, O, A, OT, \pi_{\text{act}}, \pi_{\text{time}}, \pi_{\text{ot}}, \pi_{\text{omap}}, \pi_{\text{vmap}}, \pi_{\text{ovmap}}, \leq),$$

where E is the event set, O the object set, A the activity set, OT the object-type set, and

$$e_1 \leq e_2 \iff \pi_{\text{time}}(e_1) \leq \pi_{\text{time}}(e_2)$$

with tie-breaking for equal timestamps.

(b) **Event encoding map.** For selected numeric and string event-attribute names K_{num} and K_{str} ,

$$x(e) := \left[\text{onehot}_A(\pi_{\text{act}}(e)) \mid (c_t(e))_{t \in OT} \mid (v_k(e))_{k \in K_{\text{num}}} \mid \bigoplus_{k \in K_{\text{str}}} \text{onehot}_{V_k \cup \{\text{UNK}\}}(\pi_{\text{vmap}}(e, k)) \right] \in \mathbb{R}^n,$$

with

$$c_t(e) := |\{o \in \pi_{\text{omap}}(e) : \pi_{\text{ot}}(o) = t\}| \in \mathbb{R}_{\geq 0}, \quad n := |A| + |OT| + |K_{\text{num}}| + \sum_{k \in K_{\text{str}}} (|V_k| + 1).$$

(c) **Per-object trailing window.** For object o and sequence $x_{1:T}^{(o)}$,

$$g_t^{(o)} := \left[x_{t-w+1}^{(o)} \mid x_{t-w+2}^{(o)} \mid \cdots \mid x_t^{(o)} \right] \in \mathbb{R}^{wn}.$$

(d) **PCA compression.** With centered data matrix $X_c \in \mathbb{R}^{m \times n}$,

$$\Sigma := \frac{1}{m} X_c^\top X_c, \quad \Sigma v_i = \lambda_i v_i, \quad z = W^\top (x - \mu), \quad \hat{x} = \mu + WW^\top (x - \mu).$$

The explained variance ratio is

$$\text{EVR}(d) := \frac{\sum_{i=1}^d \lambda_i}{\sum_{i=1}^n \lambda_i}.$$

(e) **SOM BMU and boundary condition.** For SOM prototypes w_i at grid coordinates r_i ,

$$b(x) := \arg \min_i \|x - w_i\|_2, \quad f(x) = r_{b(x)}.$$

For event-aligned state sequence $r_t^{(o)} = f(z_t^{(o)})$, a transition occurs when

$$r_{t-1}^{(o)} \neq r_t^{(o)}.$$

The paper records the boundary window via

$$\text{if } r_{t-1}^{(o)} = i, \ r_t^{(o)} = j \neq i, \quad B_i^{\text{exit}} \leftarrow B_i^{\text{exit}} \cup \{g_t^{(o)}\}, \quad B_j^{\text{enter}} \leftarrow B_j^{\text{enter}} \cup \{g_t^{(o)}\}.$$

Task 2: Event encoding by hand

Consider activities $A = \{\text{Create, Approve, Ship}\}$, object types $OT = \{\text{Order, Item}\}$, one numeric attribute $K_{\text{num}} = \{\text{amount}\}$, and one string attribute $K_{\text{str}} = \{\text{channel}\}$ with $V_{\text{channel}} = \{\text{web, phone}\}$ plus UNK.

Event	Activity	Related objects	amount	channel
e_1	Create	$\{o_1 : \text{Order}, i_1 : \text{Item}\}$	100	web
e_2	Create	$\{o_2 : \text{Order}, i_2 : \text{Item}, i_3 : \text{Item}\}$	240	phone
e_3	Approve	$\{o_1 : \text{Order}\}$	100	web
e_4	Ship	$\{o_1 : \text{Order}, i_1 : \text{Item}\}$	100	web
e_5	Approve	$\{o_2 : \text{Order}\}$	240	phone
e_6	Ship	$\{o_2 : \text{Order}, i_2 : \text{Item}, i_3 : \text{Item}\}$	240	email

Use block order

$$[\text{Create, Approve, Ship} \mid \# \text{Order}, \# \text{Item} \mid \text{amount} \mid \text{web, phone, UNK}].$$

For each event $e_1, \dots, e_6$, compute the four blocks separately and then write the full concatenated vector. Treat the unseen value email as UNK. Use this output format for every event:

$$x(e_1) = [\dots \mid \dots \mid \dots \mid \dots].$$

Task 3: Encoding block roles

Create a four-row table with columns *block*, *information encoded*, *why the block is needed*, and *what is lost if the block is omitted*. Include one row each for activity one-hot, object-type counts, numeric attributes, and string attributes. After the table, answer these three questions:

- What process information is lost if the object-type count block $(c_t(e))_{t \in OT}$ is removed?
- Which block would capture a stock level in the paper's inventory case study?
- Which blocks remain directly usable in traditional event logs, where every event belongs to one case?

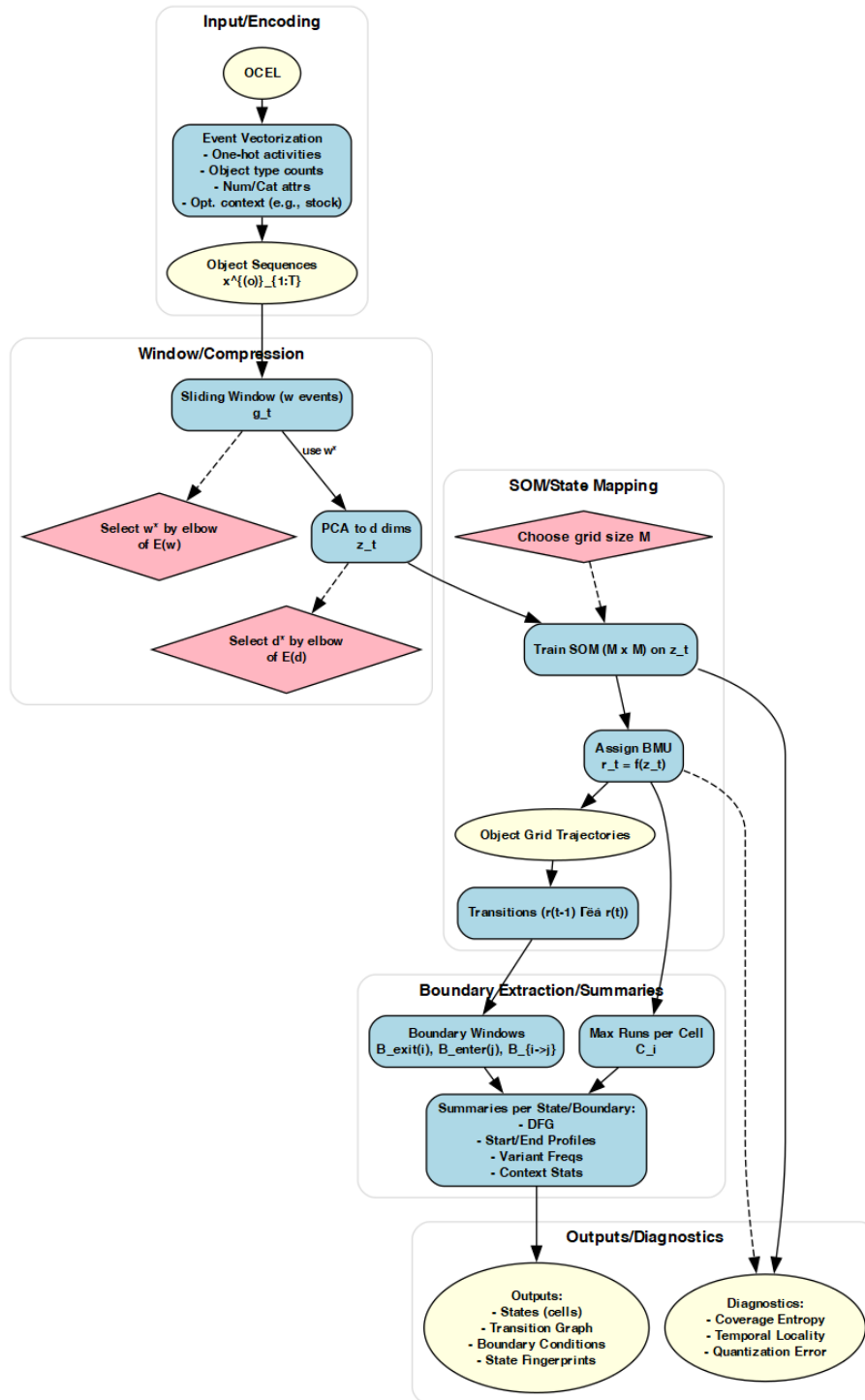


Figure 1: Pipeline from the base paper: window, compress, map, then summarize states and boundaries. Extracted from the paper PDF.

2 Sliding Windows

Task 4: Window definitions

For an object o with ordered vectors $x_{1:T}^{(o)}$, the paper uses the trailing concatenation window

$$g_t^{(o)} = \left[x_{t-w+1}^{(o)} \mid x_{t-w+2}^{(o)} \mid \cdots \mid x_t^{(o)} \right] \in \mathbb{R}^{wn}, \quad t \geq w.$$

Write the analogous trailing-window formula for a traditional event log case c , using vectors $x_{1:T}^{(c)}$. Then write two short comparison bullets:

- **Concatenation:** preserves order and positional information, but grows the dimension to wn .
- **Mean aggregation:** maps the same window to $\bar{g}_t^{(o)} = \frac{1}{w} \sum_{j=t-w+1}^t x_j^{(o)} \in \mathbb{R}^n$, losing within-window order but staying compact.

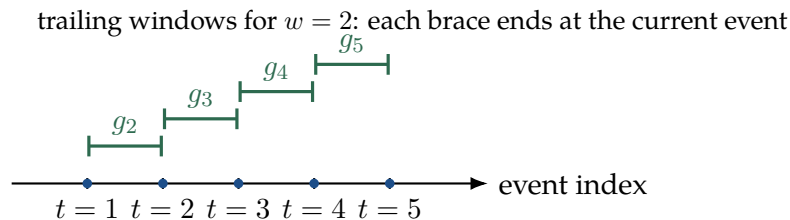
Finish with one concrete thesis scenario where concatenation is preferable and one concrete thesis scenario where mean aggregation is preferable.

Task 5: Window computation

A single object has five events at $t = 1, 2, 3, 4, 5$ with

$$x_1 = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad x_4 = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}, \quad x_5 = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}.$$

- For $w = 1, 2, 3$, list all admissible concatenated vectors g_t .
- For $w = 1, 2, 3$, compute the mean-aggregated versions $\bar{g}_t$.
- Draw the $w = 2$ trailing windows on the timeline below.



Task 6: Window function

Implement `compute_windows(event_vectors, w)`. It should return concatenated trailing windows for every event position, using a zero vector for missing early context.

```

1 import numpy as np
2
3 def compute_windows(event_vectors: np.ndarray, w: int) -> np.ndarray:
4     """Return one concatenated trailing window per event.
5
6     Parameters
7     -----
8     event_vectors:
9         Array of shape (T, n).
10
11     w:
12         Number of events per trailing window.
13
14     Returns
15     """

```

```

14  -----
15  windows:
16      Array of shape (T, w*n). Early windows are left-padded with zeros.
17  """
18  # TODO: implement
19  raise NotImplementedError
20
21  X = np.array([[1, 0, 2],
22               [0, 1, 1],
23               [1, 1, 0],
24               [0, 2, 1],
25               [2, 0, 1]], dtype=float)
26
27  print(compute_windows(X, w=2))

```

Test on the hand-computation example above. Print the returned matrix for $w = 2$ and verify that the first row is $[0, 0, 0 \mid 1, 0, 2]$ and the last row is $[0, 2, 1 \mid 2, 0, 1]$.

Task 7: Window-size elbow

The paper forms candidate window datasets in $\mathbb{R}^{wn}$, applies PCA at a fixed probe dimension, and computes a reconstruction error curve $E(w)$. Provide three answers: describe the elbow point geometrically, state why $E(w)$ usually decreases as w grows, and give one reason why an overly large w can still hurt state discovery.

3 PCA – Compress

Task 8: PCA formulas

Given centered data $X_c \in \mathbb{R}^{m \times n}$, define

$$\Sigma = \frac{1}{m} X_c^\top X_c, \quad \Sigma v_i = \lambda_i v_i, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0,$$

with orthonormal eigenvectors v_i . PCA selects $W = [v_1, \dots, v_d]$ and maps

$$z = W^\top (x - \mu), \quad \hat{x} = \mu + Wz = \mu + WW^\top (x - \mu).$$

For one point,

$$e(x) = \|x - \hat{x}\|_2^2,$$

and for a dataset the average squared reconstruction error is

$$\frac{1}{m} \sum_{j=1}^m \|x^{(j)} - \hat{x}^{(j)}\|_2^2.$$

The explained variance ratio is

$$\text{EVR}(d) = \frac{\sum_{i=1}^d \lambda_i}{\sum_{i=1}^n \lambda_i}.$$

After the formulas, write the PCA optimality claim exactly: among all rank- d linear reconstructions, PCA minimizes the average squared reconstruction error. Then add two sentences explaining why this compression step is useful before SOM training.

Task 9: PCA by hand

Use the four centered data points

$$(-2, -1), \quad (-1, 0), \quad (1, 0), \quad (2, 1).$$

- (a) Compute the covariance matrix $\Sigma = \frac{1}{4} X^\top X$.
- (b) Find the eigenvalues and eigenvectors analytically.
- (c) Project all four points to 1-D along the leading eigenvector v_1 .
- (d) Compute $\text{EVR}(1)$.
- (e) Sketch the points, the v_1 direction, and the projection of $(2, 1)$.

Task 10: Elbow rule on eigenvalues

Let the PCA eigenvalues be

$$\lambda = (9, 6, 1, 0.4, 0.1).$$

For $d = 1, \dots, 4$, compute residual variance

$$E(d) = \sum_{i=d+1}^5 \lambda_i.$$

Normalize the curve using the paper's rule

$$\tilde{d} = \frac{d-1}{d_{\max}-1}, \quad \tilde{E}(d) = \frac{E(d) - E(d_{\max})}{E(1) - E(d_{\max})}.$$

The reference line from $(0, 1)$ to $(1, 0)$ is $\tilde{d} + \tilde{E} - 1 = 0$, so select

$$\hat{d} = \arg \max_d \frac{|\tilde{d} + \tilde{E}(d) - 1|}{\sqrt{2}}.$$

Draw the scree plot and mark the selected $\hat{d}$.

Task 11: PCA implementation

Implement `pca_reduce(X, d)`. Then apply it to a random 50×8 matrix and plot $\text{EVR}(d)$ for $d = 1, \dots, 8$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def pca_reduce(X: np.ndarray, d: int):
5     """Center X, eigendecompose covariance, and return Z plus EVR(d)."""
6     # TODO:
7     # 1. compute mu and centered Xc
8     # 2. compute Sigma = Xc.T @ Xc / len(X)
9     # 3. use np.linalg.eigh, sort descending
10    # 4. project to top d components
11    # 5. return Z, evr
12    raise NotImplementedError
13
14 rng = np.random.default_rng(7)
15 X = rng.normal(size=(50, 8))

```

Report the returned shape of Z , the EVR for $d = 2$, and one sentence comparing the EVR curve with the reconstruction-error intuition from the previous task.

Task 12: Intra-case PCA hypothesis

Write a structured answer with three paragraphs: how intra-case prefix vectors differ from pooled OCEL object windows, whether this should increase or decrease the expected number of PCA components, and which empirical diagnostic you would inspect to confirm the choice of d .

4 Self-Organizing Maps

Task 13: SOM formulas

An SOM has prototypes $\{w_i\}_{i=1}^{M_x M_y}$, $w_i \in \mathbb{R}^d$, located at grid coordinates

$$G = \{(p, q) : p = 1, \dots, M_x, q = 1, \dots, M_y\}.$$

For input $z \in \mathbb{R}^d$, the best-matching unit is

$$b(z) = \arg \min_i \|z - w_i\|_2, \quad f(z) = r_{b(z)}.$$

During training, for selected input z ,

$$w_i(t+1) = w_i(t) + \eta(t) h_{b,i}(t)(z - w_i(t)),$$

where

$$h_{b,i}(t) = \exp \left(-\frac{\|r_b - r_i\|_2^2}{2\sigma(t)^2} \right).$$

After the update rule, state the topology-preservation property in one precise sentence and state why it makes state trajectories interpretable.

Task 14: SOM update

Use the following grid coordinates and prototypes:

$$w_{(0,0)} = \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix}, \quad w_{(0,1)} = \begin{bmatrix} 0.3 \\ 0.9 \end{bmatrix}, \quad w_{(1,0)} = \begin{bmatrix} 0.9 \\ 0.2 \end{bmatrix}, \quad w_{(1,1)} = \begin{bmatrix} 0.8 \\ 0.8 \end{bmatrix}.$$

For input $z = (0.8, 0.3)^\top$, learning rate $\eta = 0.5$, and Gaussian neighborhood $\sigma = 1$:

- Compute the Euclidean distance from z to each prototype and find the BMU.
- Compute $h_{b,i}$ for each neuron using grid distance.
- Update every prototype with $w_i^+ = w_i + \eta h_{b,i}(z - w_i)$.
- Plot old and new prototype positions in the 2-D input space.

Task 15: Grid-size score

For $M = 2$ and $M = 3$, use the occupancy counts below. Let N be total assigned points, $p_i = n_i/N$,

$$H(M) = -\sum_i p_i \log p_i, \quad \tilde{H}(M) = \frac{H(M)}{\log(M^2)}.$$

The neighbor-preservation rates are given as $R(2) = 0.92$ and $R(3) = 0.81$. Compute

$$J(M) = \alpha \tilde{H}(M) + (1 - \alpha)R(M), \quad \alpha = 0.5.$$

Which M wins?

Grid	Occupancy counts
$M = 2$	(35, 30, 25, 10)
$M = 3$	(18, 15, 13, 12, 11, 10, 9, 7, 5)

Task 16: SOM training plot

Train a 3×3 SOM on 100 random 3-D points. Use minisom if installed; otherwise implement the BMU and update rule manually.

- Plot a U-matrix heatmap: each cell's average prototype distance to neighboring cells.
- Plot the trajectory of 10 sequential points on the grid.
- Save both plots and note whether adjacent time points tend to remain close.

```

1 rng = np.random.default_rng(12)
2 X = rng.normal(size=(100, 3))
3
4 # TODO: train a 3x3 SOM and compute bmu positions:
5 # bmus = [(row, col), ...]
```

Task 17: SOM transition graph

Using Figure 2, write four short answers:

- Which cells look most stable, based on self-loops or large assigned-window counts?
- Which transitions appear most frequent?
- What would a boundary condition look like on this graph?
- Use the blank grid to sketch a simplified version of the most important transitions.

(0, 2)	(1, 2)	(2, 2)
(0, 1)	(1, 1)	(2, 1)
(0, 0)	(1, 0)	(2, 0)

blank 3×3 SOM grid

5 Boundary Conditions

Task 18: Boundary notation

For trained SOM states $r_t^{(o)} = f(z_t^{(o)})$, define a transition at (o, t) whenever $r_{t-1}^{(o)} \neq r_t^{(o)}$. The corresponding boundary window is

$$g_t^{(o)} = (e_{t-w^*+1}^{(o)}, \dots, e_t^{(o)}).$$

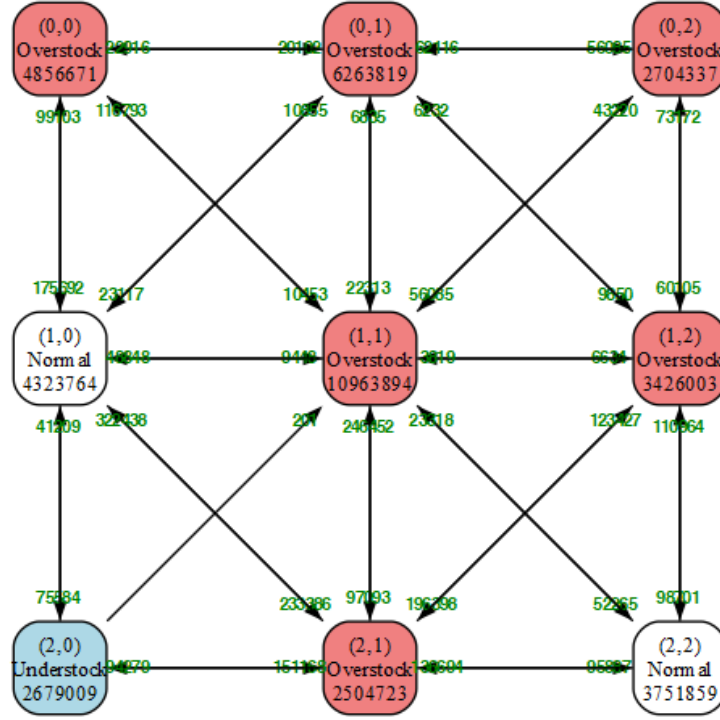


Figure 2: State-transition view of the paper's 3×3 SOM. Extracted from the paper PDF.

Boundary collections are updated as

$$\text{if } r_{t-1}^{(o)} = i, r_t^{(o)} = j \neq i, \quad B_i^{\text{exit}} \leftarrow B_i^{\text{exit}} \cup \{g_t^{(o)}\}, \quad B_j^{\text{enter}} \leftarrow B_j^{\text{enter}} \cup \{g_t^{(o)}\}.$$

Within-state behavior for state i is collected from maximal intervals

$$I_i^{(o)} = \{[a, b] : w^* \leq a \leq b \leq T^{(o)}, r_t^{(o)} = i \forall t \in [a, b], r_{a-1}^{(o)} \neq i, r_{b+1}^{(o)} \neq i \text{ when defined}\}.$$

Then C_i contains the event-identifier sequences wholly inside state i . State exactly what evidence goes into C_i , B_i^{exit} , B_j^{enter} , and $B_{i \rightarrow j}$. Finish with two sentences describing what differences between these collections reveal.

Task 19: Boundary windows in a toy trajectory

Given state trajectory

$$\langle A, A, A, B, B, A, A, C \rangle$$

and event features:

t	State	Activity	Feature note
1	A	Submit	normal arrival
2	A	Check	waiting low
3	A	Rework	defect found
4	B	Rework	defect persists
5	B	Approve	manager intervention
6	A	Check	returned to normal
7	A	Approve	cleared
8	C	Escalate	service-level risk

(a) List all transition points.

- (b) For horizon $h = 1$, list the boundary windows around each transition, using one event before and one event after the boundary when available.
- (c) For the $A \rightarrow B$ transition, compare boundary windows against intra-state A windows. Which activity is most enriched near the boundary?

Task 20: Resource-state boundaries

Define at least three resource-boundary summaries for the thesis. For each summary, specify the input data, the formula or counting rule, and what a high value means at the boundary of a resource-state transition. Consider active cases per resource, handover patterns, queue length, waiting time, activity specialization, and sudden disappearance or replacement of a resource.

6 Three State Types for Traditional Event Logs

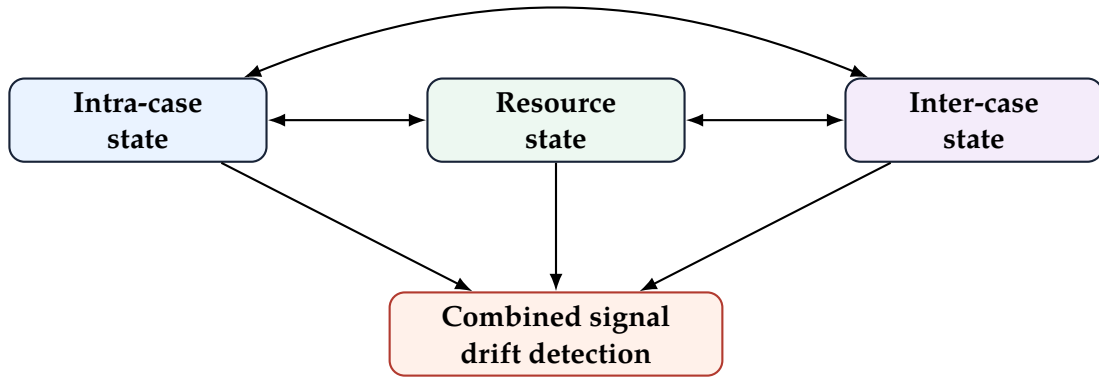


Figure 3: Three complementary state views for classical event logs.

6.1 Intra-Case State

Task 21: Intra-case definition

Write the intra-case state definition for event position t of case c as a compact representation of the running prefix $\langle e_1^{(c)}, \dots, e_t^{(c)} \rangle$, mapped through PCA and a SOM to a discrete state:

$$x_t^{\text{intra}} = \phi_{\text{intra}}(e_1^{(c)}, \dots, e_t^{(c)}), \quad s_t^{\text{intra}} = f_{\text{SOM}}^{\text{intra}}(W_{\text{intra}}^{\top}(x_t^{\text{intra}} - \mu_{\text{intra}})).$$

After the formula, list the feature vector components you would include:

- activity one-hot counts in the prefix,
- time since first event,
- time since last event,
- remaining-time estimate,
- case attributes.

Finish with exactly two differences from the paper's per-object OCEL windows.

Task 22: Intra-case feature table

One case has five events:

$$\begin{aligned} e_1 &= (\text{Submit}, \text{Alice}, 08:00), \\ e_2 &= (\text{Check}, \text{Bob}, 08:45), \\ e_3 &= (\text{Approve}, \text{Alice}, 10:00), \\ e_4 &= (\text{Notify}, \text{Carol}, 10:10), \\ e_5 &= (\text{Close}, \text{Bob}, 11:00). \end{aligned}$$

Use activity order

$$\mathcal{A} = \{\text{Submit}, \text{Check}, \text{Approve}, \text{Notify}, \text{Close}\}.$$

For each event e_t , compute:

- (a) prefix activity-count vector over $\mathcal{A}$,
- (b) elapsed time since first event in minutes,
- (c) time since last event Δt in minutes, using 0 for the first event,
- (d) $x_t^{\text{intra}} \in \mathbb{R}^7$ as [five counts | elapsed | Δt].

t	Activity	Resource	elapsed	Δt	x_t^{intra}
1	Submit	Alice			
2	Check	Bob			
3	Approve	Alice			
4	Notify	Carol			
5	Close	Bob			

Task 23: Intra-case windows with $w = 2$

Using the five x_t^{intra} vectors from the previous task, compute concatenated windows

$$\bar{x}_t^{\text{intra}} = [x_{t-1}^{\text{intra}} | x_t^{\text{intra}}],$$

with a zero vector for $t = 1$. Which window captures the most activity diversity? Define your criterion explicitly, for example number of nonzero activity-count dimensions across the window.

6.2 Resource State

Task 24: Resource-state definition

Write the resource-state definition at timestamp τ as a global representation of resource workload and interaction patterns across all cases:

$$x_\tau^{\text{res}} = \phi_{\text{res}}(L_{\leq \tau}, \Delta T), \quad s_\tau^{\text{res}} = f_{\text{SOM}}^{\text{res}}(W_{\text{res}}^\top (x_\tau^{\text{res}} - \mu_{\text{res}})).$$

Then list the candidate features you would compute: workload per resource, active cases per resource, handover frequency, average waiting time, activity distribution per resource, and role assignments. Finish with one concrete resource drift that would not change the activity sequence distribution.

Task 25: Resource feature table

Use this 9-event log:

CaseID	Activity	Timestamp	Resource
C1	Submit	08:00	Alice
C2	Submit	08:05	Alice
C1	Check	08:20	Bob
C3	Submit	08:25	Carol
C2	Check	08:35	Bob
C1	Approve	08:50	Alice
C3	Check	09:00	Bob
C2	Approve	09:10	Carol
C3	Approve	09:25	Carol

At $\tau \in \{08:20, 08:35, 08:50, 09:10, 09:25\}$:

- Count active cases per resource. A case is active after its first event and before its final event.
- Compute handover rate: among consecutive event pairs inside cases observed up to τ , what proportion has a different resource?
- Build $x_\tau^{\text{res}} = [\text{activeAlice}, \text{activeBob}, \text{activeCarol}, \text{handoverRate}]$.
- With a global window width $\Delta T = 30$ minutes, aggregate resource vectors over $[\tau - \Delta T, \tau]$. Describe how the vector changes when a burst of new cases starts.

τ	Alice	Bob	Carol	handover	x_τ^{res} after 30-min aggregation
08:20					
08:35					
08:50					
09:10					
09:25					

6.3 Inter-Case State

Task 26: Inter-case definition

Write the inter-case state definition at timestamp τ as a global temporal behavior representation:

$$x_\tau^{\text{inter}} = \phi_{\text{inter}}(L_{\leq \tau}, \Delta T), \quad s_\tau^{\text{inter}} = f_{\text{SOM}}^{\text{inter}}(W_{\text{inter}}^\top (x_\tau^{\text{inter}} - \mu_{\text{inter}})).$$

Then list the candidate features you would compute: number of active cases, inter-arrival time between new cases, global throughput in events per time unit, and proportion of cases in each current activity. Finish with one example feature pattern for congestion, one for a burst, and one for a slow phase.

Task 27: Inter-case feature table

Using the same 9-event log and timestamps $\tau \in \{08:20, 08:35, 08:50, 09:10, 09:25\}$:

- Count total active cases.
- Compute the mean inter-arrival gap between case starts up to τ .
- Compute throughput: number of events in the last 30 minutes.
- Assemble $x_\tau^{\text{inter}} = [\text{activeCases}, \text{meanInterArrivalGap}, \text{throughput30min}]$.

τ	active cases	mean inter-arrival gap	throughput	x_τ^{inter}
08:20				
08:35				
08:50				
09:10				
09:25				

Task 28: Inter-case SOM assignment

Assign the five inter-case vectors from the previous task to a 2×2 SOM with prototypes:

$$w_{(0,0)} = (1, 20, 1), \quad w_{(0,1)} = (2, 15, 2), \quad w_{(1,0)} = (2, 10, 3), \quad w_{(1,1)} = (3, 8, 4).$$

Scale each feature to comparable units before computing Euclidean distances. For each timestamp, report the closest prototype, draw the resulting grid trajectory, and list the state-transition timestamps. Then give one concrete example from the 9-event log showing how a shared resource or batch-processing constraint appears in the inter-case features.

7 Combined State Signal & Concept Drift Detection**Task 29: Combined state signal**

At each timestamp τ , define the combined state signal as

$$S_\tau = (s_\tau^{\text{intra}}, s_\tau^{\text{res}}, s_\tau^{\text{inter}}).$$

Give at least three scalar transformations of this triple for drift detection. For each transformation, write either a formula or clear pseudocode:

- novelty indicator: 1 if the joint state was rare or unseen in a reference window,
- distribution distance: total variation, Jensen-Shannon distance, or Earth mover's distance between sliding-window state distributions,
- trajectory score: distance between mean SOM positions before and after a candidate split,
- transition intensity: number of state changes per unit time.

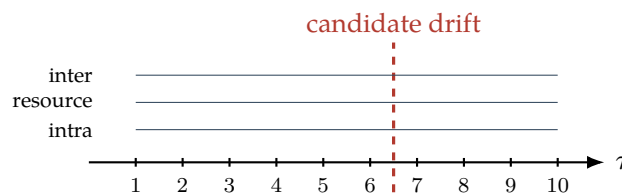
Choose one scalar for the thesis evaluation and justify the choice in two sentences.

Task 30: State-stream drift analysis

A process runs for 10 timestamps:

$$\begin{aligned} s^{\text{intra}} &= \langle A, A, A, A, B, B, B, C, C, C \rangle, \\ s^{\text{res}} &= \langle 1, 1, 1, 2, 2, 2, 2, 3, 3 \rangle, \\ s^{\text{inter}} &= \langle \alpha, \alpha, \beta, \beta, \beta, \beta, \gamma, \gamma, \gamma, \gamma \rangle. \end{aligned}$$

- At which timestamps does each individual sequence change?
- Construct the joint sequence S_τ .
- Plot each sequence on the timeline and mark candidate drift points.
- At which timestamp does the combined signal most clearly indicate drift?



Task 31: Window distribution shift

Two time windows yield the following joint-state frequencies:

Joint state	freq _{W₁}	freq _{W₂}
(A, 1, α)	0.50	0.10
(A, 2, β)	0.25	0.20
(B, 2, β)	0.20	0.35
(C, 3, γ)	0.05	0.35

- (a) Which state has the largest relative frequency shift?
 (b) Compute total variation distance:

$$\text{TV}(P, Q) = \frac{1}{2} \sum_s |P(s) - Q(s)|.$$

- (c) Is this sudden or gradual drift? State what additional temporal evidence you would need.

Task 32: SOM trajectory shift

For 20 events, the state trajectory is

(0, 0), (0, 0), (0, 1), (0, 1), (1, 1), (1, 1), (1, 1), (1, 2), (1, 2), (1, 2),
 (2, 1), (2, 1), (2, 2), (2, 2), (2, 2), (2, 2), (2, 1), (2, 1), (2, 0), (2, 0).

- (a) Draw the trajectory on the 3×3 grid.
 (b) Compute the mean position before and after event 10.
 (c) Has the center of mass shifted? What process change could this indicate?
 (d) Define and compute the drift score

$$\|\bar{r}_{\text{before}} - \bar{r}_{\text{after}}\|_2.$$

(0, 2)	(1, 2)	(2, 2)
(0, 1)	(1, 1)	(2, 1)
(0, 0)	(1, 0)	(2, 0)

blank 3×3 SOM grid

Task 33: Sliding-window distribution detector

Generate 100 rows with columns timestamp, intra_state, res_state, and inter_state. Inject a known drift at row 50. Then:

- (a) Plot each state stream as a step function.
 (b) Compute a sliding-window joint-state distribution with window size 10.
 (c) Flag timestamps where the distribution shift exceeds a threshold.

```

1 from collections import Counter
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 def state_distribution(states, start, width):
7     """Return normalized Counter over states[start:start+width]."""
8     # TODO
9     raise NotImplementedError
10
11 def total_variation(p, q):
12     """Compute TV distance between two dict-like distributions."""
13     # TODO
14     raise NotImplementedError
15
16 # TODO: simulate pre-drift and post-drift regimes, then detect row 50.

```

Task 34: Resource-drift experiment

Design one experiment showing that the three-SOM signal detects a resource drift that ProDrift or CV4CDD would miss. Specify all four items below:

- injected drift, such as Bob leaving at time τ^* and Dave taking over Bob's checks with longer waiting times,
- held-constant behavior, such as the same activity sequence distribution before and after drift,
- evaluation metric, such as detection delay, F1 over drift windows, or localization accuracy,
- expected outcome for the three-SOM signal versus control-flow-only baselines.

8 Implementation Roadmap

Task 35: Pipeline skeleton

Complete the Python skeleton below. Keep the function signatures, replace each placeholder exception with pseudocode-level steps, and write one docstring per function:

```

1 def load_log(path: str):
2     """Load a traditional event log as a dataframe with case, activity,
3     timestamp, and optional resource/attribute columns."""
4     raise NotImplementedError
5
6 def encode_events(df, feature_config: dict):
7     """Create event-level vectors for intra-case, resource, and inter-case
8     feature families."""
9     raise NotImplementedError
10
11 def compute_windows(event_vectors, w: int, mode: str = "concat"):
12     """Build trailing windows using concatenation or aggregation."""
13     raise NotImplementedError
14
15 def fit_pca(X, d: int):
16     """Fit PCA, returning projected vectors plus model parameters."""
17     raise NotImplementedError
18
19 def fit_som(Z, grid_shape=(3, 3), random_state: int = 0):
20     """Train a SOM on PCA vectors."""
21     raise NotImplementedError
22

```



```

23 def assign_states(Z, som):
24     """Map each vector to its SOM best-matching unit."""
25     raise NotImplementedError
26
27 def detect_drift(state_stream, window: int, threshold: float):
28     """Compare sliding-window state distributions and return alarms."""
29     raise NotImplementedError

```

After the code block, add a small table with one row per function: inputs, outputs, and one unit-test idea.

Task 36: PM4PY intra-case PCA

PM4PY's local documentation states that `tree_generator.apply()` generates a process tree and that `process_tree.playout.apply(tree, parameters={"num_traces": ...})` performs a process-tree playout returning an event log. Use that API to create a sample log, convert it to a dataframe, compute intra-case feature vectors, and visualize their 2-D PCA projection.

```

1 from pm4py.algo.simulation.tree_generator import algorithm as tree_generator
2 from pm4py.algo.simulation.playout.process_tree import algorithm as tree_playout
3 import pm4py
4
5 tree = tree_generator.apply(parameters={"mode": 8, "min": 5, "max": 12})
6 log = tree_playout.apply(tree, parameters={"num_traces": 200})
7 df = pm4py.convert_to_dataframe(log)
8
9 # TODO:
10 # 1. ensure case/activity/timestamp columns are available
11 # 2. compute prefix activity-count vectors per case
12 # 3. append elapsed time and time since previous event
13 # 4. PCA to 2D
14 # 5. scatter plot, color by case length bucket

```

If the generated playout lacks timestamps, add synthetic timestamps per case before computing temporal features.

Task 37: Paper implementation reuse

Inspect the paper's [GitHub repository](#). Produce a reuse map with four rows:

- Which scripts or modules can be reused directly for windowing, PCA, SOM training, state assignment, or visualization?
- Which parts are OCEL-specific and must be rewritten for traditional event logs?
- Where should the thesis implementation add resource-state feature builders and inter-case-state feature builders?
- What tests would demonstrate equivalence to the paper's pipeline on the shared components?